YAYASAN MINAR SORTA TIGA PUTRI

POLITEKNIK BISNIS INDONESIA

Disusun Oleh:

Nama: Rulli Anaya Fajduani

NIM: 25110129

Mata Kuliah: Praktek

Dosen Pembimbing: Juni Ismail

PENJELASAN LENGKAP REPOSITORY PRAKTEK06

Praktikum Java JDBC dan Database Inventori Laboratorium

Daftar Isi

- 1. Gambaran Umum Proyek
- 2. Struktur Folder dan File
- 3. Alur Kerja Program
- 4. Penjelasan KoneksiDB.java
- 5. Penjelasan UjiKoneksi.java
- 6. Penjelasan Database dan SQL
- 7. File Konfigurasi dan Library
- 8. Cara Menjalankan Proyek
- 9. Skenario Error dan Cara Mengatasinya
- 10. Pertanyaan Dosen dan Jawaban
- 11. Kesimpulan

Catatan: daftar isi ini disusun secara manual agar mudah dibaca dan diedit.

1. Gambaran Umum Proyek

Repository praktek06 berisi proyek Java untuk menguji koneksi JDBC ke database inventori laboratorium. Fokus utamanya adalah memeriksa apakah aplikasi Java dapat membaca konfigurasi, memuat driver JDBC, terhubung ke server MySQL/MariaDB, menjalankan query sederhana, dan menutup koneksi dengan benar.

JDBC (Java Database Connectivity) adalah API Java yang memungkinkan program Java berkomunikasi dengan database. MySQL Connector/J merupakan driver yang menerjemahkan permintaan JDBC agar dapat digunakan untuk berkomunikasi dengan server MySQL yang kompatibel.

Gambaran alur koneksi

1. Program Java dijalankan dari NetBeans.
2. KoneksiDB membaca file config/database.properties.
3. Program memuat kelas driver com.mysql.cj.jdbc.Driver.
4. DriverManager mencoba membuka koneksi ke server database.
5. UjiKoneksi menjalankan query SELECT COUNT(*) FROM barang.
6. Program menampilkan hasil pengujian dan memastikan koneksi ditutup.

Proyek ini berfokus pada pengujian koneksi dan query sederhana. Kode yang dijelaskan bukan aplikasi inventori lengkap dengan fitur antarmuka untuk menambah, mengubah, dan menghapus barang.

2. Struktur Folder dan File

File/Folder	Fungsi
praktek06-main/	Folder utama repository hasil ekstraksi ZIP.
README.md	Berkas pengantar. Pada repository ini, isinya hanya judul praktek06 sehingga dokumentasinya masih singkat.
P06_01_berhasil.png.png	Dokumentasi skenario koneksi berhasil.
P06_02_E1_mysql_mati.png.png	Dokumentasi skenario ketika layanan MySQL mati/tidak dapat dihubungi.
P06_03_E6_password_salah.png.png	Dokumentasi skenario password salah atau autentikasi ditolak.
P06_04_E10_driver_hilang.png.png	Dokumentasi skenario ketika driver JDBC tidak ditemukan.
bahan/db_inventori_lab.sql	File SQL untuk memasukkan data awal ke database inventori. Periksa struktur tabel karena file ini berisi data awal dan tidak menyediakan seluruh definisi CREATE TABLE.
bahan/mysql-connector-j-26.7.0/	Folder distribusi driver MySQL Connector/J, dokumentasi, dan file pendukung.
proyek/PrKoneksiInventori/	Proyek Java yang dibuka dengan NetBeans.
src/koneksi/KoneksiDB.java	Membaca konfigurasi, memuat driver, membuat URL, dan membuka koneksi database.
src/koneksi/UjiKoneksi.java	Kelas utama yang menjalankan tahapan pengujian dan melaporkan kesalahan.
config/database.example.properties	Contoh konfigurasi. Perlu disalin/dijadikan database.properties dan disesuaikan.
nbproject/	Pengaturan proyek NetBeans, termasuk properti build dan referensi library.

build.xml	Skrip Apache Ant untuk membangun dan menjalankan proyek.
manifest.mf	Metadata untuk arsip JAR.
.gitignore	Daftar file/folder yang tidak perlu dilacak oleh Git.
.gitattributes	Pengaturan perlakuan Git terhadap jenis file.
LICENSE	Lisensi MIT yang menyertai repository.

3. Alur Kerja Program

Secara sederhana, prosesnya dapat dipahami sebagai berikut:

7. Konfigurasi: program membaca alamat server, port, nama database, username, dan password.
8. Driver: Java memuat MySQL Connector/J agar dapat menggunakan JDBC.
9. Koneksi: DriverManager mencoba membuka koneksi dengan URL, username, dan password.
10. Query: program menjalankan SELECT COUNT(*) FROM barang untuk menghitung jumlah baris tabel barang.
11. Penutupan: koneksi ditutup menggunakan try-with-resources, lalu program memeriksa statusnya.

Jika salah satu tahap gagal, program menampilkan tahap yang gagal dan saran pemeriksaan.

4. Penjelasan KoneksiDB.java

Lokasi: proyek/PrKoneksiInventori/src/koneksi/KoneksiDB.java

4.1 Package dan import

```
package koneksi;
```

Menempatkan kelas di dalam package bernama koneksi. Package membantu mengelompokkan kelas yang berkaitan.

```
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;
```

IOException menangani masalah pembacaan file; InputStream membaca data dari file; Files, Path, dan Paths membantu mengakses lokasi file; Connection mewakili koneksi database; DriverManager membuat koneksi; SQLException mewakili kesalahan database; Properties menyimpan pasangan kunci dan nilai konfigurasi.

4.2 Deklarasi kelas dan driver

```
public final class KoneksiDB {
    public static final String DRIVER =
            "com.mysql.cj.jdbc.Driver";
}
```

public berarti dapat diakses dari kelas lain. final mencegah kelas diwariskan. DRIVER menyimpan nama kelas driver MySQL Connector/J. static membuat konstanta bisa diakses melalui kelas, sedangkan final menandakan nilainya tetap.

4.3 Lokasi file konfigurasi

```
public static final Path FILE_KONFIGURASI =
        Paths.get("config", "database.properties");
```

Program mencari konfigurasi relatif terhadap direktori kerja saat program dijalankan. Karena itu, lokasi kerja dan struktur folder harus sesuai agar file dapat ditemukan.

4.4 Kunci konfigurasi wajib

```
private static final String[] KUNCI_WAJIB = {
    "db.host", "db.port", "db.name", "db.user"
};
```

Array ini berisi empat nama pengaturan yang wajib diisi. Program memeriksa setiap nilai agar tidak kosong. Password tidak tercantum di daftar wajib; bila server membutuhkan password, password yang salah atau kosong dapat menyebabkan penolakan autentikasi.

4.5 Konstruktor privat

```
private KoneksiDB() {
}
```

Konstruktor privat mencegah kelas lain membuat objek KoneksiDB. Kelas ini dirancang berisi metode utilitas yang dipanggil langsung, seperti KoneksiDB.getConnection().

4.6 Membaca konfigurasi

```
public static Properties bacaKonfigurasi()
        throws SQLException {
    Path lokasi = FILE_KONFIGURASI.toAbsolutePath();
    Properties konfigurasi = new Properties();

    try (InputStream input = Files.newInputStream(lokasi)) {
        konfigurasi.load(input);
    } catch (IOException e) {
        throw new SQLException(
            "File konfigurasi tidak ditemukan atau tidak dapat dibaca: "
            + lokasi, e);
    }

    for (String kunci : KUNCI_WAJIB) {
        if (konfigurasi.getProperty(kunci, "").trim().isEmpty()) {
            throw new SQLException(
                "Nilai " + kunci + " wajib diisi pada file " + lokasi);
        }
    }

    String port = konfigurasi.getProperty("db.port").trim();
    if (!port.matches("\\d{1,5}")) {
        throw new SQLException("Nilai db.port harus berupa angka.");
    }
    return konfigurasi;
}
```

Metode ini membentuk alamat absolut file, membaca properti, memeriksa empat nilai wajib, dan memvalidasi port sebagai satu sampai lima digit angka. Validasi ini hanya memeriksa bentuk teks; belum menjamin port tersedia atau berada dalam rentang port yang valid.

try-with-resources menutup InputStream secara otomatis. Jika file tidak dapat dibaca, IOException ditangkap dan dibungkus sebagai SQLException dengan pesan yang lebih mudah dipahami.

4.7 Membuat URL JDBC

```
public static String buatUrl(Properties konfigurasi) {
    return "jdbc:mysql://"
            + konfigurasi.getProperty("db.host").trim() + ":"
            + konfigurasi.getProperty("db.port").trim() + "/"
            + konfigurasi.getProperty("db.name").trim()
            + "?connectTimeout=5000&socketTimeout=10000";
}
```

URL menyatukan host, port, nama database, dan batas waktu. connectTimeout=5000 mengatur batas waktu koneksi sekitar 5 detik; socketTimeout=10000 mengatur batas waktu komunikasi socket sekitar 10 detik.

4.8 Memuat driver

```
public static void muatDriver() throws SQLException {
    try {
        Class.forName(DRIVER);
    } catch (ClassNotFoundException e) {
        throw new SQLException(
            "Driver JDBC tidak ditemukan. Tambahkan JAR driver ke Libraries proyek.", e);
    }
}
```

Class.forName meminta Java memuat kelas driver. Jika kelas tidak tersedia pada classpath, ClassNotFoundException ditangkap lalu diteruskan sebagai SQLException dengan pesan petunjuk.

4.9 Membuka koneksi

```
public static Connection getConnection() throws SQLException {
    Properties konfigurasi = bacaKonfigurasi();
    muatDriver();

    return DriverManager.getConnection(
        buatUrl(konfigurasi),
        konfigurasi.getProperty("db.user").trim(),
        konfigurasi.getProperty("db.password", ""));
}
```

Metode ini membaca konfigurasi, memuat driver, membuat URL, lalu meminta DriverManager membuka koneksi. Jika berhasil, metode mengembalikan objek Connection. Jika gagal, SQLException diteruskan ke pemanggil.

5. Penjelasan UjiKoneksi.java

Lokasi: proyek/PrKoneksiInventori/src/koneksi/UjiKoneksi.java

5.1 Import dan metode main

File ini menggunakan kelas untuk menangani koneksi, metadata database, query SQL, hasil query, serta beberapa jenis kesalahan jaringan. Metode main adalah titik awal eksekusi program.

```
public static void main(String[] args) {
    int tahap = 0;
    Connection koneksiTerpakai = null;
```

```
    System.out.println("=== UJI KONEKSI JDBC - PRAKTIKUM 6 ===");
}
```

Variabel tahap mencatat tahapan yang sedang dilakukan. koneksiTerpakai disiapkan untuk menyimpan koneksi yang telah digunakan. System.out.println menampilkan teks ke jendela Output.

5.2 Tahap 1 — Membaca konfigurasi

```
tahap = 1;
Properties konfigurasi = KoneksiDB.bacaKonfigurasi();
String password = konfigurasi.getProperty("db.password", "");
```

Program membaca pengaturan dari file. Password dibaca untuk memeriksa statusnya, tetapi isi password tidak seharusnya dicetak. Jika file hilang atau pengaturan wajib kosong, program gagal pada tahap ini.

5.3 Tahap 2 — Memuat driver

```
tahap = 2;
KoneksiDB.muatDriver();
```

Tahap ini memeriksa apakah kelas driver tersedia. Driver berhasil dimuat bukan berarti server database sudah dapat dihubungi; koneksi server baru dicoba pada tahap berikutnya.

5.4 Tahap 3 — Membuka koneksi

```
tahap = 3;
long mulai = System.currentTimeMillis();

try (Connection con = KoneksiDB.getConnection()) {
    koneksiTerpakai = con;
    long durasi = System.currentTimeMillis() - mulai;
    DatabaseMetaData meta = con.getMetaData();
}
```

currentTimeMillis mencatat waktu. Selisih waktunya digunakan untuk mengukur durasi koneksi. getMetaData mengambil informasi server dan driver, seperti nama serta versi database. try-with-resources menutup koneksi secara otomatis ketika blok selesai.

5.5 Tahap 4 — Query verifikasi

```
String sql = "SELECT COUNT(*) AS jumlah FROM barang";

try (PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
    rs.next();
    System.out.println("Hasil: " + rs.getInt("jumlah") + " barang");
}
```

SELECT COUNT(*) menghitung baris tabel barang. AS jumlah memberi nama pada kolom hasil. PreparedStatement menyiapkan query, executeQuery menjalankannya, rs.next berpindah ke baris hasil pertama, dan getInt membaca nilai jumlah sebagai bilangan bulat. Query ini memerlukan tabel barang yang sudah dibuat.

5.6 Tahap 5 — Menutup koneksi

```
System.out.println(
    "Koneksi sudah ditutup? " + koneksiTerpakai.isClosed());
```

isClosed mengembalikan true jika koneksi telah ditutup dan false jika masih terbuka. Pemeriksaan harus dilakukan setelah blok try-with-resources selesai agar status penutupan dapat terlihat.

5.7 Menangani kesalahan

```
} catch (SQLException e) {
    laporkanGagal(tahap, e);
}
```

Jika terjadi SQLException, program memanggil laporkanGagal untuk menampilkan tahap kegagalan, SQLState, kode error, pesan, penyebab dasar, dan saran perbaikan.

5.8 Metode laporkanGagal dan beriSaran

```
Throwable akar = e;
while (akar.getCause() != null) {
    akar = akar.getCause();
}
```

Perulangan menelusuri rantai penyebab kesalahan sampai tidak ada cause berikutnya. Metode beriSaran kemudian menggunakan tahap dan informasi error untuk menyarankan pemeriksaan konfigurasi, driver, host, autentikasi, database, tabel, atau hak akses.

6. Penjelasan Database dan SQL

Database db_inventori_lab ditujukan untuk menyimpan data inventori laboratorium. File SQL yang tersedia digunakan untuk memasukkan data awal. Pastikan struktur tabel telah dibuat sebelum menjalankan perintah INSERT.

Nama Tabel	Fungsi
barang	Menyimpan data barang, misalnya kode_barang, nama_barang, id_kategori, jumlah_tersedia, dan keterangan.
kategori	Mengelompokkan barang, misalnya perangkat input, kabel dan konektor, perangkat jaringan, serta alat ukur.
peminjaman	Menyimpan informasi transaksi peminjaman, peminjam, tanggal, dan status.
detail_peminjaman	Mencatat barang dan jumlahnya pada masing-masing transaksi. Satu transaksi dapat memiliki beberapa baris detail.
pengguna	Menyimpan informasi akun, nama pengguna, peran, status aktif, dan kolom password hash.

Contoh INSERT

```
INSERT INTO barang
(kode_barang, nama_barang, id_kategori, jumlah_tersedia, keterangan)
VALUES
('BRG-001', 'Keyboard USB', 1, 10, 'Rak A1');
```

INSERT INTO memasukkan baris baru ke tabel. Daftar pertama adalah kolom tujuan; VALUES berisi nilai dalam urutan yang sama. Pastikan tipe data dan relasi foreign key sesuai dengan struktur tabel.

Transaksi SQL

```
SET AUTOCOMMIT = 0;
START TRANSACTION;
-- perintah pengisian data
COMMIT;
```

AUTOCOMMIT = 0 menonaktifkan commit otomatis pada sesi. START TRANSACTION memulai transaksi dan COMMIT mengonfirmasi perubahan. Jika terjadi kegagalan, transaksi dapat memerlukan ROLLBACK sesuai kebutuhan.

Catatan data pengguna: nilai placeholder seperti BELUM_DIATUR_P14 pada kolom password_hash bukan hash password yang siap dipakai untuk login. Sistem autentikasi nyata harus memakai proses hashing yang aman.

7. File Konfigurasi dan Library

7.1 Contoh database.properties

```
db.host=127.0.0.1
db.port=3306
db.name=db_inventori_lab
db.user=root
db.password=
```

Contoh tersebut hanya cocok jika MySQL lokal memakai user root tanpa password. Jika menggunakan akun lain, ubah nilai sesuai konfigurasi. Jangan membagikan file konfigurasi yang berisi password asli.

7.2 Referensi driver di NetBeans

Properti proyek merujuk ke file JAR pada alamat Windows tertentu. Alamat itu mungkin tidak ada pada laptop lain. Buka Project Properties → Libraries dan tambahkan JAR dari folder bahan. Pastikan versi dan lokasi file sesuai.

7.3 File proyek lainnya

build.xml

Skrip Ant untuk proses build, kompilasi, dan menjalankan proyek.

nbproject/project.xml dan project.properties

Menyimpan metadata proyek dan pengaturan build, termasuk kelas utama serta referensi library.

manifest.mf

Menyimpan metadata untuk arsip JAR.

.gitignore

Menghindari file hasil build atau file lokal tertentu ikut dilacak Git.

.gitattributes

Mengatur perlakuan Git terhadap file tertentu.

LICENSE

Menyatakan ketentuan lisensi untuk penggunaan dan distribusi repository.

8. Cara Menjalankan Proyek

12. Ekstrak praktek06-main.zip ke folder yang mudah ditemukan.
13. Buka NetBeans dan pilih File → Open Project.
14. Pilih folder proyek proyek/PrKoneksiInventori.
15. Pastikan MySQL/MariaDB aktif, misalnya melalui XAMPP jika digunakan.
16. Buat database db_inventori_lab.
17. Pastikan kelima tabel beserta kolom dan relasinya telah dibuat. File SQL yang tersedia berisi data awal, bukan seluruh definisi CREATE TABLE.
18. Impor bahan/db_inventori_lab.sql untuk memasukkan data awal. Periksa apakah data sudah ada agar tidak terjadi duplikasi primary key.
19. Buat folder config di direktori kerja yang dipakai program, lalu salin database.example.properties menjadi database.properties.
20. Isi host, port, nama database, username, dan password sesuai pengaturan MySQL lokal.
21. Di NetBeans, klik kanan proyek → Properties → Libraries. Perbaiki referensi dan tambahkan file JAR MySQL Connector/J jika belum terdaftar.
22. Buka UjiKoneksi.java dan pilih Run File, atau gunakan Shift+F6.
23. Baca jendela Output untuk melihat tahap yang berhasil atau pesan error.

9. Skenario Error dan Cara Mengatasinya

Masalah	Pemeriksaan/solusi
File konfigurasi tidak ditemukan	Pastikan config/database.properties ada pada direktori kerja program dan nama file tepat.
Nilai konfigurasi kosong	Isi db.host, db.port, db.name, dan db.user. Periksa salah ketik pada nama properti.
Driver tidak ditemukan	Tambahkan file JAR MySQL Connector/J ke Libraries/classpath dan pastikan versi file cocok.
MySQL mati atau koneksi ditolak	Jalankan layanan database, periksa host dan port, lalu pastikan firewall tidak memblokir koneksi.
Password salah atau akses ditolak	Periksa username, password, metode autentikasi, dan hak akses akun.
Database tidak ditemukan	Pastikan database db_inventori_lab telah dibuat dan ejaan nama benar.
Tabel barang tidak ditemukan	Buat struktur tabel yang diperlukan sebelum menjalankan file data SQL.
Query ditolak karena hak akses	Pastikan akun memiliki izin SELECT pada tabel yang diperlukan.

10. Pertanyaan Dosen dan Jawaban

Apa tujuan Praktikum 6?

Menguji koneksi Java ke MySQL/MariaDB menggunakan JDBC dan mempelajari cara mengenali kesalahan koneksi.

Apa itu JDBC?

Java Database Connectivity, yaitu API Java untuk berkomunikasi dengan database.

Apa fungsi MySQL Connector/J?

Driver yang memungkinkan Java memakai JDBC untuk berkomunikasi dengan server MySQL.

Apa perbedaan KoneksiDB dan UjiKoneksi?

KoneksiDB mengurus konfigurasi dan membuka koneksi; UjiKoneksi menjalankan tahapan pengujian dan melaporkan hasil.

Apa fungsi SELECT COUNT(*) FROM barang?

Menghitung jumlah baris pada tabel barang dan memverifikasi bahwa query dapat dijalankan.

Mengapa menggunakan try-with-resources?

Agar resource seperti InputStream, Connection, PreparedStatement, dan ResultSet ditutup otomatis ketika blok selesai.

Apa perbedaan driver hilang dan MySQL mati?

Driver hilang membuat Java gagal memuat kelas driver pada tahap 2. MySQL mati membuat upaya koneksi ke server gagal pada tahap 3.

Mengapa konfigurasi disimpan terpisah dari kode?

Agar pengaturan lebih mudah diganti tanpa mengedit source code. File tetap harus dijaga agar password tidak bocor.

Apa arti isClosed() bernilai true?

Objek Connection menyatakan bahwa koneksi sudah ditutup.

Apakah program ini sudah menjadi aplikasi inventori lengkap?

Belum. Kode Java yang diuji berfokus pada koneksi dan query hitung baris; bukan aplikasi CRUD lengkap.

11. Kesimpulan

Repository praktek06 memperlihatkan dasar koneksi database dari Java. KoneksiDB.java menangani konfigurasi, driver, URL, dan pembukaan koneksi. UjiKoneksi.java menguji koneksi, membaca metadata, menjalankan query, menutup koneksi, dan memberikan saran jika terjadi kesalahan. File SQL menyediakan data awal inventori, sedangkan gambar mendokumentasikan skenario uji.

Sebelum praktik, periksa tiga hal utama: file config/database.properties harus tersedia, struktur database dan tabel harus sudah dibuat, serta referensi JAR driver harus sesuai dengan lokasi di laptop. Dengan persiapan itu, hasil pengujian akan lebih mudah dipahami dan kesalahan lebih mudah ditelusuri.